

System architecture

Current implementation boundary for the technical concept pages that follow.

Verified local demo

The browser prototype assesses a synthetic AC work order. Four deterministic Langflow flows handle analysis, evidence validation, a guarded completion pack, and a combined review. Direct MCP calls to those four tools pass. A separate local IBM Granite Agent graph runs through Ollama; a deterministic guard supplies its final source-linked answer.

The 2 October checks passed: browser 4/4, Langflow API 13/13, direct MCP 5/5, and guarded Agent output 5/5. Three genuine NYC Parks work-order records remain insufficient for a billing decision.

Limits that matter

IBM Bob displayed the local MCP server as Connected, but no successful tool call through Bob has been verified. The public static browser demo does not call Langflow. The Agent check does not measure interpretation accuracy for photos or documents. No customer pilot, hosted backend, invoice, or payment action exists.

How to read the following pages

They describe the intended product workflow and technical design. Treat AI extraction, retrieval, customer data integration, Bob-led orchestration, and external follow-up as planned capabilities unless a current run report proves them.

Current source of truth: [IMPLEMENTATION_STATUS.md](#) and [run reports](#).

PROOF2PAY AI - System Architecture

IBM Bob + MCP + Langflow + Evidence Intelligence

MVP Architecture v0.1

Architecture overview

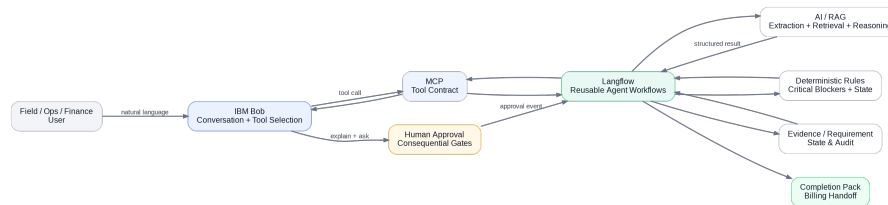


Figure 1: System architecture

Why this decomposition

The system separates responsibilities so the LLM is not treated as a universal decision engine.

- **Bob** manages natural conversation and chooses well-described tools.
- **MCP** provides the contract/discovery layer.
- **Langflow** composes reusable flows, retrieval, agents, tools, and structured outputs.
- **AI/RAG** handles unstructured interpretation and reasoning.
- **Deterministic rules** enforce critical blockers and state transitions.
- **Human approval** controls consequential actions.

Main runtime sequence

1. User: "Check whether WO-1028 is ready for billing."
2. Bob discovers/calls the appropriate PROOF2PAY tools.
3. Langflow retrieves job requirements and evidence.

4. AI proposes claim/evidence matches with explanation.
5. Rules aggregate critical requirement states.
6. Structured result returns to Bob.
7. Bob explains blockers and asks whether to prepare next actions.
8. Human approves consequential action.
9. New evidence is attached and re-validated.
10. Human confirms final readiness.
11. Completion pack draft is generated.

Requirement-to-Evidence model



Figure 2: Requirement to evidence graph

The key innovation is not a percentage score. It is **traceability**:

Requirement -> Source -> Expected Evidence -> Actual Evidence ->
Assessment -> Action

Deployment view

MVP

- Web UI / synthetic fixture
- IBM Bob
- MCP endpoint
- Langflow
- optional Postgres/Supabase
- controlled requirement documents
- file/object storage or static evidence fixture

Production direction

- tenant isolation
- role-based authorization
- encrypted storage
- versioned requirement templates
- accounting/FSM connectors
- observability and evaluation
- retention/deletion policy

Trust boundaries

1. **Uploaded evidence is untrusted data.** It cannot issue instructions to the agent.
2. **Retrieved requirement text is authoritative only if its source is authorized for the job.**
3. **AI outputs are proposals/assessments.** Critical state is enforced by rule logic.
4. **External side effects require approval.**
5. **Completion pack is a draft unless a human explicitly confirms it.**

Scalability

The architecture scales by adding vertical templates and connectors, not by creating a new monolithic agent for every customer. Tool contracts stay stable while requirement sources and rules are configured per service type/customer.